

Time Series Models Snippets

1. Stationarity

```
In [84]: # Import packages

import pandas as pd
import numpy as np
from statsmodels.tsa.stattools import adfuller
import statsmodels.api as sm
from statsmodels.tsa.stattools import coint
import matplotlib.pyplot as plt
import os
```

```
In [5]: # Read commodities dataset

path = ""
commodities_data = pd.read_csv("commodity_futures.csv")
```

```
In [7]: # Take a Look

commodities_data.head(10)
```

```
Out[7]:
```

	Date	CLc1	Cc1	HGc1	LCOc1	NGc1	Wc1
0	2014-12-31	53.27	3.9700	2.8385	57.33	2.889	5.8975
1	2015-01-02	52.69	3.9575	2.8385	56.42	3.003	5.8125
2	2015-01-05	50.04	4.0600	2.7930	53.11	2.882	5.8900
3	2015-01-06	47.93	4.0500	2.8040	51.10	2.938	5.9175
4	2015-01-07	48.65	3.9625	2.7960	51.15	2.871	5.7950
5	2015-01-08	48.79	3.9425	2.8070	50.96	2.927	5.6700
6	2015-01-09	48.36	4.0025	2.7905	50.11	2.946	5.6375
7	2015-01-12	46.07	4.0200	2.7630	47.43	2.795	5.5550
8	2015-01-13	45.89	3.8575	2.6790	46.59	2.943	5.4800
9	2015-01-14	48.48	3.8100	2.5530	48.69	3.233	5.3775

```
In [9]: commodities_data.tail(10)
```

Out[9]:

	Date	CLc1	Cc1	HGc1	LCOc1	NGc1	Wc1
2811	2025-11-17	59.91	4.3475	5.0000	64.20	4.361	5.4425
2812	2025-11-18	60.74	4.3675	4.9625	64.89	4.371	5.4650
2813	2025-11-19	59.44	4.2975	5.0080	63.51	4.550	5.3675
2814	2025-11-20	59.14	4.2650	4.9595	63.38	4.474	5.2700
2815	2025-11-21	58.06	4.2550	5.0085	62.56	4.580	5.2700
2816	2025-11-24	58.84	4.2375	4.9620	63.37	4.549	5.2225
2817	2025-11-25	57.95	4.2350	4.9995	62.48	4.424	5.2725
2818	2025-11-26	58.65	4.3175	5.1045	63.13	4.558	5.2900
2819	2025-11-27	58.65	4.3175	5.1045	63.34	4.558	5.2900
2820	2025-11-28	58.55	4.3550	5.1855	63.20	4.850	5.3100

This is a front-end commodity futures contracts dataset. I think it should be interesting to work on.

In [12]: *# Set index, datetime*

```
commodities = commodities_data.copy()
commodities["Date"] = pd.to_datetime(commodities["Date"])
commodities = commodities.set_index("Date")
```

In [14]: *# Generate returns because time series is most likely stationary*

```
prices = commodities.copy()

returns = commodities.pct_change()
returns = returns.dropna()

log_returns = np.log(commodities / commodities.shift(1))
log_returns = log_returns.replace(
    [np.inf, -np.inf],
    np.nan
)
log_returns = log_returns.dropna()
```

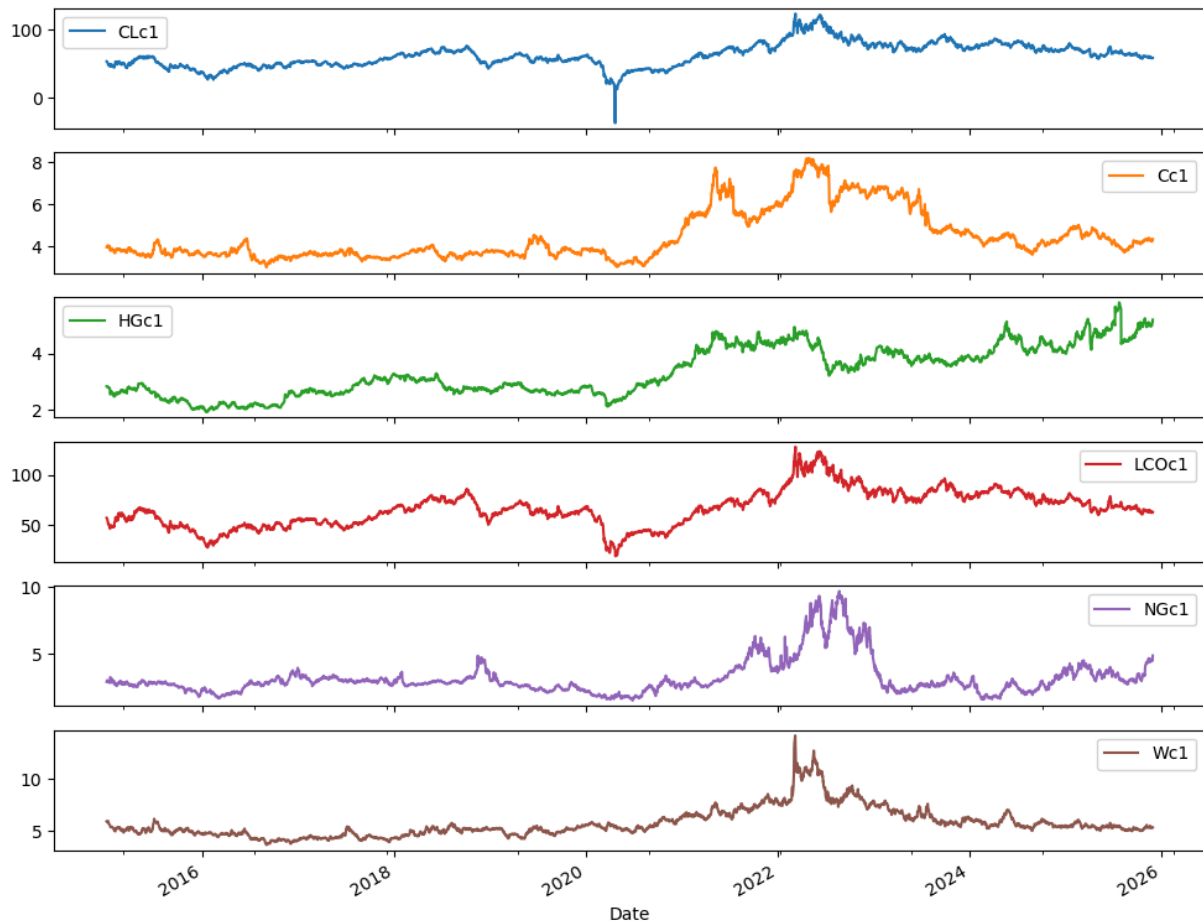
C:\Users\pcarg\anaconda3\Lib\site-packages\pandas\core\internals\blocks.py:393: RuntimeWarning: invalid value encountered in log
 result = func(self.values, **kwargs)

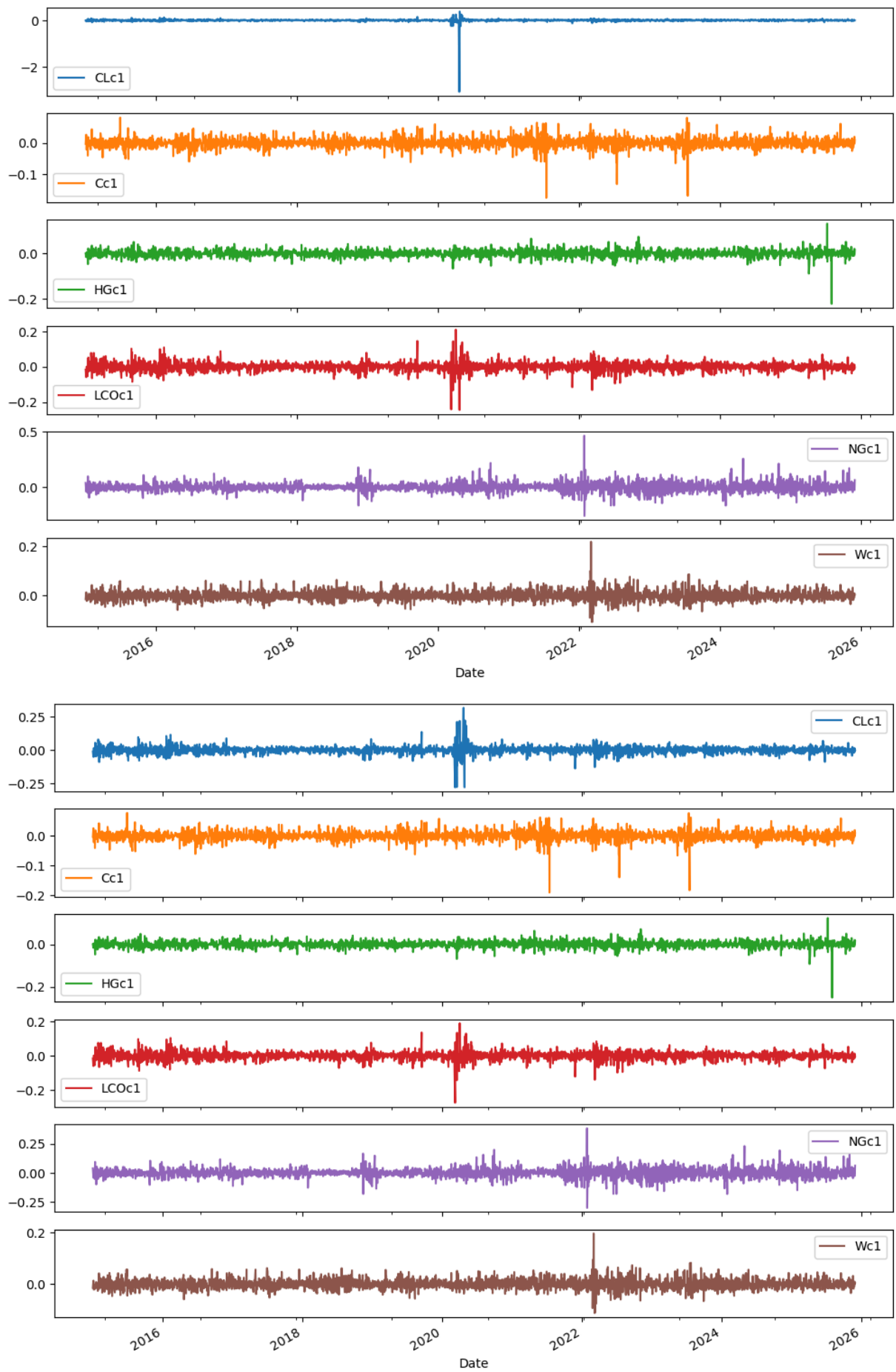
In [16]: *# Let's plot to see what we have*

```
prices.plot(
    subplots=True,
    figsize=(12,10)
)
```

```
returns.plot(  
    subplots=True,  
    figsize=(12,10)  
)  
  
log_returns.plot(  
    subplots=True,  
    figsize=(12,10)  
)
```

Out[16]: array([<Axes: xlabel='Date'>, <Axes: xlabel='Date'>,
 <Axes: xlabel='Date'>, <Axes: xlabel='Date'>,
 <Axes: xlabel='Date'>, <Axes: xlabel='Date'>], dtype=object)





In [22]: *# Let's see which time series is the most volatile*

```
print(returns.std().sort_values())
```

```
HGc1    0.014834
Cc1     0.015717
Wc1     0.019473
LCOc1   0.024580
NGc1    0.039328
CLc1    0.068513
dtype: float64
```

In [24]: `print(log_returns.std().sort_values())`

```
HGc1    0.014976
Cc1     0.015861
Wc1     0.019357
LCOc1   0.024166
CLc1    0.028177
NGc1    0.038928
dtype: float64
```

Commodity prices often behave approximately as unit-root processes because shocks to supply, demand, inventories, geopolitics, and macro conditions can have persistent effects on the price level. Returns are much closer to stationary because they measure incremental changes rather than accumulated levels. A trader typically prefers working with returns because many statistical models assume stationarity. Using non-stationary price series can lead to spurious regressions, invalid inference, and misleading estimates of predictability.

In [29]: *# Let's run ADF test on prices*

```
for col in commodities.columns:

    result = adfuller(commodities[col].dropna())

    print(
        col,
        "ADF =", round(result[0], 3),
        "p =", round(result[1], 5)
    )
```

```
CLc1 ADF = -2.397 p = 0.1426
Cc1 ADF = -1.708 p = 0.42712
HGc1 ADF = -0.812 p = 0.81557
LCOc1 ADF = -2.142 p = 0.22783
NGc1 ADF = -2.832 p = 0.05381
Wc1 ADF = -2.223 p = 0.19788
```

In [31]: *# Now on returns*

```
for col in returns.columns:

    result = adfuller(returns[col].dropna())

    print(
```

```
col,
    "ADF =", round(result[0], 3),
    "p =", round(result[1], 5)
)
```

```
CLc1 ADF = -9.779 p = 0.0
Cc1 ADF = -24.459 p = 0.0
HGc1 ADF = -55.078 p = 0.0
LCOc1 ADF = -52.118 p = 0.0
NGc1 ADF = -22.975 p = 0.0
Wc1 ADF = -25.328 p = 0.0
```

In [33]: *# Now on Log returns*

```
for col in log_returns.columns:

    result = adfuller(log_returns[col].dropna())

    print(
        col,
        "ADF =", round(result[0], 3),
        "p =", round(result[1], 5)
    )
```

```
CLc1 ADF = -10.119 p = 0.0
Cc1 ADF = -24.555 p = 0.0
HGc1 ADF = -54.879 p = 0.0
LCOc1 ADF = -52.444 p = 0.0
NGc1 ADF = -24.613 p = 0.0
Wc1 ADF = -25.367 p = 0.0
```

Why is the ADF Test One-Sided (Or left-tailed)?

The Augmented Dickey-Fuller (ADF) test is designed to detect whether a time series contains a unit root.

Starting from an AR(1) process:

$$y_t = \phi y_{t-1} + \epsilon_t$$

Subtracting y_{t-1} from both sides gives:

$$\Delta y_t = (\phi - 1)y_{t-1} + \epsilon_t$$

Defining:

$$\gamma = \phi - 1$$

we obtain:

$$\Delta y_t = \gamma y_{t-1} + \epsilon_t$$

The ADF test is therefore equivalent to testing:

$$H_0 : \gamma = 0$$

against

$$H_1 : \gamma < 0$$

Since:

$$\gamma = \phi - 1$$

the hypotheses can also be written as:

$$H_0 : \phi = 1$$

(Unit Root / Non-Stationary)

versus

$$H_1 : \phi < 1$$

(Mean-Reverting / Stationary)

The test is one-sided because it is specifically designed to distinguish a random walk from a stationary process. It is not intended to detect explosive processes ($\phi > 1$).

Why are ADF Statistics Negative?

The ADF statistic is essentially a t-statistic for the estimated coefficient ($\hat{\gamma}$):

$$ADF \approx \frac{\hat{\gamma}}{SE(\hat{\gamma})}$$

For a stationary process:

$$\phi < 1$$

which implies:

$$\gamma = \phi - 1 < 0$$

Therefore, evidence in favor of stationarity corresponds to increasingly negative estimates of γ , producing increasingly negative ADF statistics.

Intuitively:

- (ADF ≈ 0): little evidence against a unit root.
- Moderately negative ADF statistic: some evidence of mean reversion.
- Very negative ADF statistic: strong evidence of stationarity.

For example:

$$ADF = -2.0$$

may not be sufficient to reject the unit-root null, whereas

$$ADF = -20$$

provides overwhelming evidence that the series is stationary.

A useful interpretation is that a negative (γ) acts as a restoring force: when the series moves above its equilibrium level, expected future changes become negative, pulling it back toward equilibrium.

2. Cointegration

Cointegration occurs when two or more non-stationary time series share a stable long-run equilibrium relationship. Individually the series may contain unit roots and drift over time, but a particular linear combination of them is stationary. Equivalently, if we regress one series on the other, the residuals are stationary and mean-reverting. This implies that although the series can wander individually, they cannot drift arbitrarily far apart in the long run.

Basically, we regress one variable on another, and see if the residuals of the regression are stationary. This is the Amish way of doing this test, which is the Engle-Granger test. It tests pairs of variables for cointegration. To be specific, it is a 2-step process, which first performs univariate OLS and then performs ADF test on the residuals to determine if they are a unit root process. The multivariate version of this problem is solved by the Johansen test.

```
In [50]: # Try this on Brent and WTI

X = commodities["CLc1"]
Y = commodities["LCOc1"]

X = sm.add_constant(X)

model = sm.OLS(Y, X).fit()

print(model.summary())

residuals = model.resid

adf = adfuller(residuals)

print("ADF:", adf[0])
print("p-value:", adf[1])
```


OLS Regression Results

=====						
Dep. Variable:	LC0c1		R-squared:	0.980		
Model:	OLS		Adj. R-squared:	0.980		
Method:	Least Squares		F-statistic:	1.380e+05		
Date:	Wed, 03 Jun 2026		Prob (F-statistic):	0.00		
Time:	21:48:45		Log-Likelihood:	-6596.2		
No. Observations:	2821		AIC:	1.320e+04		
Df Residuals:	2819		BIC:	1.321e+04		
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	3.8592	0.175	21.992	0.000	3.515	4.203
CLc1	1.0075	0.003	371.493	0.000	1.002	1.013
=====						
Omnibus:	3257.841		Durbin-Watson:	0.379		
Prob(Omnibus):	0.000		Jarque-Bera (JB):	1499692.986		
Skew:	5.343		Prob(JB):	0.00		
Kurtosis:	115.448		Cond. No.	240.		
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

ADF: -5.490108642786843

p-value: 2.184936346492788e-06

```
In [53]: print(model.params)
         print(model.rsquared)
```

```
const    3.85917
CLc1     1.00754
dtype: float64
0.979982410415962
```

The intercept is the average premium of Brent over WTI. This is economics: I think basically because WTI is landlocked. Before we get into Johansen and linear algebra, let's run Engle-Granger tests in all potential combinations of our commodity time series.

```
In [60]: cols = commodities.columns

results = pd.DataFrame(
    index=cols,
    columns=cols
)

for i in cols:
    for j in cols:

        if i == j:
            results.loc[i,j] = None

        else:
```

```

        score, pvalue, _ = coint(
            commodities[i],
            commodities[j]
        )

        results.loc[i,j] = pvalue

print(results)

```

	CLc1	Cc1	HGc1	LCOc1	NGc1	Wc1
CLc1	None	0.022048	0.217313	0.000016	0.016788	0.025213
Cc1	0.048468	None	0.613813	0.065861	0.121139	0.000325
HGc1	0.624595	0.896158	None	0.859127	0.746405	0.910507
LCOc1	0.000019	0.027428	0.384075	None	0.036118	0.019779
NGc1	0.005594	0.004295	0.050882	0.007752	None	0.008556
Wc1	0.017129	0.000088	0.442812	0.026036	0.038798	None

```

In [64]: cointegrated_pairs = []

for i in range(len(cols)):
    for j in range(i + 1, len(cols)):

        pvalue = float(results.iloc[i, j])

        if pvalue < 0.05:

            cointegrated_pairs.append(
                (
                    cols[i],
                    cols[j],
                    pvalue
                )
            )

cointegrated_pairs = sorted(
    cointegrated_pairs,
    key=lambda x: x[2]
)

print("Cointegrated pairs (p < 0.05):\n")

for pair in cointegrated_pairs:
    print(
        f"{pair[0]} ↔ {pair[1]} | p-value = {pair[2]:.6f}"
    )

```

Cointegrated pairs ($p < 0.05$):

CLc1 ↔ LC0c1 | p-value = 0.000016
 Cc1 ↔ Wc1 | p-value = 0.000325
 NGc1 ↔ Wc1 | p-value = 0.008556
 CLc1 ↔ NGc1 | p-value = 0.016788
 LC0c1 ↔ Wc1 | p-value = 0.019779
 CLc1 ↔ Cc1 | p-value = 0.022048
 CLc1 ↔ Wc1 | p-value = 0.025213
 LC0c1 ↔ NGc1 | p-value = 0.036118

Now that we know that these time series are cointegrated, let's take a look at stationary spread.

```
In [68]: for pair in cointegrated_pairs:

    x_col = pair[0]
    y_col = pair[1]

    X = sm.add_constant(
        commodities[x_col]
    )

    Y = commodities[y_col]

    model = sm.OLS(
        Y,
        X
    ).fit()

    alpha = model.params["const"]
    beta = model.params[x_col]

    print("\n=====")
    print(f"{y_col} = {alpha:.4f} + {beta:.4f} * {x_col}")

    print(
        f"Stationary spread:"
    )

    print(
        f"{y_col} - ({alpha:.4f} + {beta:.4f} * {x_col})"
```

```
=====
LCOc1 = 3.8592 + 1.0075 * CLc1
Stationary spread:
LCOc1 - (3.8592 + 1.0075 * CLc1)
```

```
=====
Wc1 = 0.8166 + 1.0947 * Cc1
Stationary spread:
Wc1 - (0.8166 + 1.0947 * Cc1)
```

```
=====
Wc1 = 3.4340 + 0.7183 * NGc1
Stationary spread:
Wc1 - (3.4340 + 0.7183 * NGc1)
```

```
=====
NGc1 = 0.2221 + 0.0472 * CLc1
Stationary spread:
NGc1 - (0.2221 + 0.0472 * CLc1)
```

```
=====
Wc1 = 1.8921 + 0.0572 * LCOc1
Stationary spread:
Wc1 - (1.8921 + 0.0572 * LCOc1)
```

```
=====
Cc1 = 1.3576 + 0.0499 * CLc1
Stationary spread:
Cc1 - (1.3576 + 0.0499 * CLc1)
```

```
=====
Wc1 = 1.9952 + 0.0595 * CLc1
Stationary spread:
Wc1 - (1.9952 + 0.0595 * CLc1)
```

```
=====
NGc1 = 0.0992 + 0.0460 * LCOc1
Stationary spread:
NGc1 - (0.0992 + 0.0460 * LCOc1)
```

```
In [70]: # Residuals

residuals = model.resid

print(
    "Spread Mean:",
    residuals.mean()
)

print(
    "Spread Std:",
    residuals.std()
)
```

```
Spread Mean: 2.418011436829834e-16
Spread Std: 1.0782682135349912
```

I hate the names of the contracts. Let's map them to actual commodities so we understand, and make some plots.

```
In [73]: name_map = {
    "CLc1": "WTI Crude Oil",
    "LCOc1": "Brent Crude Oil",
    "NGc1": "Natural Gas",
    "HGc1": "Copper",
    "Cc1": "Corn",
    "Wc1": "Wheat"
}
```

```
In [ ]: # =====
# Generate charts for all cointegrated pairs
# =====

for pair in cointegrated_pairs:

    x_col = pair[0]
    y_col = pair[1]

    x_name = name_map[x_col]
    y_name = name_map[y_col]

    X = commodities[x_col]
    Y = commodities[y_col]

    X_reg = sm.add_constant(X)

    model = sm.OLS(
        Y,
        X_reg
    ).fit()

    residuals = model.resid

    spread_z = (
        residuals - residuals.mean()
    ) / residuals.std()

    pair_name = (
        f"{y_name}_vs_{x_name}"
        .replace(" ", "_")
    )

    # =====
    # Scatter Plot
    # =====

    plt.figure(figsize=(8, 6))

    plt.scatter(
        X,
        Y,
```

```

        alpha=0.4
    )

    plt.plot(
        X,
        model.predict(X_reg),
        linewidth=2
    )

    plt.xlabel(x_name)
    plt.ylabel(y_name)

    plt.title(
        f"{y_name} vs {x_name}"
    )

    plt.tight_layout()

    plt.savefig(
        os.path.join(
            output_folder,
            f"{pair_name}_scatter.png"
        ),
        dpi=300,
        bbox_inches="tight"
    )

    plt.close()

    # =====
    # Spread Plot
    # =====

    plt.figure(figsize=(12, 4))

    plt.plot(
        commodities.index,
        residuals
    )

    plt.axhline(
        0,
        linestyle="--"
    )

    plt.title(
        f"Cointegration Spread\n"
        f"{y_name} vs {x_name}"
    )

    plt.ylabel("Spread")

    plt.tight_layout()

    plt.savefig(
        os.path.join(

```

```

        output_folder,
        f"{pair_name}_spread.png"
    ),
    dpi=300,
    bbox_inches="tight"
)

plt.close()

# =====
# Z-Score Plot
# =====

plt.figure(figsize=(12, 4))

plt.plot(
    commodities.index,
    spread_z
)

plt.axhline(
    0,
    linestyle="--"
)

plt.axhline(
    2,
    linestyle="--"
)

plt.axhline(
    -2,
    linestyle="--"
)

plt.title(
    f"Spread Z-Score\n"
    f"{y_name} vs {x_name}"
)

plt.ylabel("Z-Score")

plt.tight_layout()

plt.savefig(
    os.path.join(
        output_folder,
        f"{pair_name}_zscore.png"
    ),
    dpi=300,
    bbox_inches="tight"
)

plt.close()

# =====

```

```

# Spread Histogram
# =====

plt.figure(figsize=(8, 5))

plt.hist(
    residuals,
    bins=50
)

plt.axvline(
    residuals.mean(),
    linestyle="--"
)

plt.title(
    f"Spread Distribution\n"
    f"{y_name} vs {x_name}"
)

plt.xlabel("Spread")

plt.ylabel("Frequency")

plt.tight_layout()

plt.savefig(
    os.path.join(
        output_folder,
        f"{pair_name}_histogram.png"
    ),
    dpi=300,
    bbox_inches="tight"
)

plt.close()

# =====
# Regression summary
# =====

alpha = model.params["const"]
beta = model.params[x_col]

print("\n" + "=" * 80)

print(
    f"{y_name} = "
    f"{alpha:.4f} + "
    f"{beta:.4f} * "
    f"{x_name}"
)

print(
    f"R2 = {model.rsquared:.4f}"
)

```



```
print(  
    f"Spread Mean = {residuals.mean():.4f}"  
)  
  
print(  
    f"Spread Std = {residuals.std():.4f}"  
)  
  
print("=" * 80)  
  
print(  
    f"\nAll charts saved in:\n{os.path.abspath(output_folder)}"  
)
```

In []: